

Test-Format Hand Cricket Discord Bot

Game-Logic Engineering Specification & Code Review

Prepared for: Development hand-off (bsw repository)

Scope: Ball-by-ball engine, no-ball / 4-zero rule, AFK handling, declarations, follow-on, multi-day play, bug fixes, performance

Constraint honoured throughout: `bot.active_matches` stays a `{channel_id: MatchInstance}` dict — all new state lives as new attributes on `MatchInstance` / its nested dicts.

0. How to use this document

This spec is organised so a developer can implement top-to-bottom: Section 1 audits what already exists, Section 2 lists bugs that will break the game loop if left unfixed, Section 3 defines the new state needed, Sections 4–11 define the actual rules engine (this is the core ask), Section 12 covers concurrency, Section 13 covers performance, and Section 14 gives a phased build order. Pseudocode is Python-flavoured and written against the existing class names (`MatchInstance`, `Game`, `GameChecks`) so it can be translated almost directly into the codebase.

Urgent — rotate your secrets before doing anything else

The uploaded project archive contains a real `.env` file with a live `BOT_TOKEN` and live `TURSO_AUTH_TOKEN` / `TURSO_DATABASE_URL` — not placeholder values. `.env` is correctly listed in `.gitignore` so it shouldn't reach GitHub, but it was bundled into the zip that left your machine. Treat both credentials as compromised: regenerate the bot token in the Discord Developer Portal, and rotate the Turso auth token (or recreate the database token) before continuing development. Anyone with the current values can fully control the bot and read/write the database. Going forward, zip only the project source (e.g. ``git archive``) instead of the whole working directory, so `.env`, `__pycache__`, and `.git` internals don't get bundled by accident.

1. Audit of the existing codebase

Before adding the test-match rules, here's what's already in place so the new logic plugs into it instead of duplicating it:

1.1 What already works

- `MatchInstance` (`core/backend/instance.py`) holds per-match state: teams, captains, toss winner, overs/days counters, `match_settings`, `players_queue`, and an unused `asyncio.Lock`.
- `bot.active_matches`: `dict[channel_id, MatchInstance]` is the single source of truth (created in `host.py`, never reset to a different structure elsewhere) — this is the dict the task says must stay untouched.
- Lobby flow is complete: `create` → `join/leave` → `lock_lobby` → `toss` (`TossView`) → `bat/bowl` choice (`BatBallView`) → `start`.
- `Game.initialise()` builds two per-player stat dictionaries (`TeamA/TeamB`) keyed by Discord user ID, each with `runs`, `balls`, `wickets`, `runs_given`, `balls_given`, `is_out`, `timeline` (a `deque(maxlen=6)`) and a few flags (`hatrick`, `duck`) that are never populated yet.
- `PlayerSelect/PlayerView` (dropdown) lets a captain pick the next batsman or bowler and writes the choice onto `match_instance.curr_batsman` / `curr_bowler`.
- `GameChecks.game_continue_check()` is the single gate that is supposed to decide whether the match keeps going.

1.2 What is missing (the actual gap you're filling)

- There is no per-ball resolution logic anywhere — no code reads a bowler's number, a batsman's number, or compares them. `Game.continue_game()` only ever asks for a new batsman/bowler in an infinite loop; nothing ever scores a run, takes a wicket, or advances overs.

- No no-ball detection, no zero/dot-ball counter, no AFK/timeout handling, no over-completion logic, no innings-transition logic, no declaration command, no follow-on logic, no day/stumps logic, no result calculation.
- timeline (the deque(maxlen=6) prepared on every player) is the obvious place to push a rolling "last 6 balls" display, but nothing pushes to it yet.
- There's no per-over state at all (legal balls bowled this over, zero-count this over, who bowled the last over) — this has to be added.

2. Bugs to fix first — these will silently break or crash the game loop

These were found while reading the code in preparation for this spec. Several of them will stop the ball-engine from ever running correctly, so fix them before wiring in new logic.

2.1 First-pass findings

B 1	CRITICAL	checks.py — GameChecks.game_continue_check()	The condition list ends with a bare <code>match_instance</code> (the object itself, not a boolean expression). A MatchInstance instance is always truthy, so <code>any(game_over_conditions)</code> is always True, meaning the function always returns False. As written, the match can never be judged "still in progress" — <code>continue_game()</code> 's while-loop will never run a single iteration.	Delete the stray <code>match_instance</code> list entry. If the intent was an AFK/abort flag, replace it with an explicit <code>match_instance.result is not None</code> check instead.
B 2	CRITICAL	checks.py — same function	<code>match_instance.score >= match_instance.target</code> is evaluated unconditionally inside the list (Python builds the whole list eagerly), even during innings 1 when target is still None. <code>int >= None</code> raises TypeError and crashes any command that calls this check.	Stop building <code>game_continue_check</code> as a flat list of pre-evaluated booleans. Convert it to short-circuited if/elif logic (see §9.4) so target is only read once it has been set, and so each end condition can carry its own result reason.
B 3	HIGH	checks.py — wickets condition	<code>match_instance.wickets >= len(match_instance.players)/2</code> divides the *combined* player pool of both teams by 2, instead of comparing the batting team's fallen wickets to the batting team's own line-up size. With uneven team sizes this ends an innings early or late.	Track wickets per-innings against the size of the batting team only (<code>squad_size - 1</code> , last-man-standing — see §9.1), not a fraction of total players.
B 4	HIGH	checks.py — <code>is_player / is_captain / is_toss_winner</code>	<code>ctx.author.id in match_instance.players</code> compares an int to a list of discord.User objects (always	Compare objects to objects (<code>ctx.author in match_instance.players</code>) or ids to ids (<code>ctx.author.id ==</code>

			False). `ctx.author.id == match_instance.teamA_captain` compares an int to a discord.User object (always False, never True). Same bug in is_toss_winner. These checks can never pass as written.	match_instance.teamA_captain.id`) consistently — pick one convention repo-wide (see §3.4).
B 5	MEDIUM	host.py — kick_player / swap_players	match_instance.players / teamA / teamB store discord.User/Member objects (see instance.py), but kick_player does `match_instance.players.remove(player.id)` and swap_players checks `player1.id in match_instance.teamA` — both compare an int against a list of User objects, so these will raise ValueError or silently no-op instead of doing anything.	Use the Member objects directly (`match_instance.players.remove(player)`), matching how join_segregate_player/leave_segregate_player already operate on objects.
B 6	MEDIUM	game.py — next_bowler()	Always offers every bowler in the bowling team's dict, including the player who just bowled the previous over. Real (and test) cricket forbids the same bowler from bowling two consecutive overs.	Filter out match_instance.last_over_bowler_id (new field, see §3.1) before building the dropdown options.
B 7	MEDIUM	instance.py — created_at	asyncio.get_event_loop().time() is deprecated outside a running loop in modern asyncio/discord.py and will warn or fail depending on version.	Use time.time() (wall clock, also easier to persist to Turso) or asyncio.get_running_loop().time() called from inside an async context.
B 8	LOW	instance.py — unused lock	self.lock = asyncio.Lock() is created but never acquired anywhere, so two near-simultaneous button presses (bowler & batsman submitting at the same time, or a double-click) can both run resolution code and double-count a ball.	Wrap the per-ball resolution critical section in `async with match_instance.lock:` (see §12).
B 9	LOW	game.py — Game.initialise()	TeamA/TeamB dicts mix player-keyed entries (int keys) with three aggregate keys ('name', 'runs', 'balls', 'wickets') stored at the same dict level. PlayerSelect already has to special-case this with isinstance(player_id, int); it's an easy place to introduce a KeyError when iterating.	Move the team-aggregate fields into their own small dict (e.g. match_instance.team_innings_stats['A']) instead of co-mingling with player rows — cheap now, painful to refactor later.

2.2 Second-pass findings (deeper read-through)

A follow-up pass through every remaining file (config loading, Turso persistence, dependency list, and the rest of the view/command flow) turned up more issues, several of them blocking too:

B1 0	CRITICAL	views/batball_view.py — bat_or_bowl_choice()	Imports <code>from core import config</code> (the module) but then reads <code>config.BOT_PREFIX</code> . The module only exposes a <code>config</code> instance with a <code>.prefix</code> attribute (see <code>core/config.py</code>) — there is no <code>BOT_PREFIX</code> name anywhere. This raises <code>AttributeError</code> the moment a captain picks Bat or Bowl, which is every single match.	Match <code>host.py</code> 's working import: <code>from core.config import config</code> , then use <code>config.prefix</code> instead of <code>config.BOT_PREFIX</code> .
B1 1	HIGH	game.py — Game.start_game()	Calls <code>await GameChecks.game_continue_check(self.match_instance)</code> but never uses or acts on the returned bool, and never calls <code>self.continue_game()</code> . Independent of bug B1, this means the play loop is never actually entered after <code>!start</code> — the method announces the match and stops.	Either call <code>await self.continue_game()</code> directly from <code>start_game()</code> , or use the check's result to decide whether to enter the loop: <code>if await GameChecks.game_continue_check(mi): await self.continue_game()</code> .
B1 2	HIGH	turso/db.py — TursoDB.init_match()	The SQL has a trailing comma before the closing parenthesis in the column list (<code>'match_stats,')</code> , the column names (<code>team_A_score, team_B_score, match_stats</code>) don't match <code>schemas.sql</code> 's actual columns (<code>team_one_score, team_two_score, match_data</code>), and the method never builds the <code>args</code> list or calls <code>self.execute(...)</code> — it just defines a string and returns. Persistence is effectively a stub, not partially working.	Rewrite <code>init_match</code> to use the real column names from <code>schemas.sql</code> , build the args list from <code>match_data</code> , and actually <code>return await self.execute(sql, args)</code> . Add matching <code>update_match / fetch_match</code> helpers while you're in there, since none exist yet.
B1 3	MEDIUM	requirements.txt	Lists <code>'libsql-client'</code> (not imported or used anywhere in the codebase) but does not list <code>'httpx'</code> , which <code>turso/db.py</code> imports directly and depends on. A fresh <code>pip install -r requirements.txt</code> will leave the bot unable to import <code>db.py</code> .	Remove the unused <code>libsql-client</code> entry and add <code>'httpx>=0.27.0'</code> (or your pinned version) to <code>requirements.txt</code> .
B1 4	MEDIUM	dropdown/player_select.py — PlayerSelect.callback()	Only stores the chosen player onto <code>match_instance.curr_batsman / curr_bowler</code> and edits the message. Nothing calls back into <code>Game</code> to continue the over (e.g. request the next ball once both batsman and bowler are confirmed). Right now picking a player is a dead end — execution doesn't resume anywhere.	Pass the live <code>Game</code> instance (or an <code>asyncio.Event / callback</code>) into <code>PlayerView/PlayerSelect</code> so the callback can invoke <code>game.request_ball()</code> once both <code>curr_batsman</code> and <code>curr_bowler</code> are set, instead of silently stopping after the dropdown closes.
B1 5	MEDIUM	host.py — change_host()	<code>'if new_host.id in match_instance.players:'</code> compares an int to a list of <code>discord.Member</code> objects — same id-vs-object mismatch as B4/B5, but in a third location they didn't cover. Will always evaluate <code>False</code> , so the host can never actually be changed.	Use <code>'if new_host in match_instance.players:'</code> per the object-comparison convention in §3.4.
B1 6	LOW	host.py — show_match_info()	Uses unrestricted <code>'getattr(match_instance, param, ...)'</code> driven directly by user input. Any player can pass arbitrary attribute names (e.g.	Replace the open <code>getattr</code> with an explicit whitelist dict mapping a small set of safe param

			<code>`b!show bot`, `b!show lock`, `b!show channel`)</code> and get internal objects — the live Bot instance, the <code>asyncio.Lock</code> , channel internals — dumped into an embed, which is both an information-disclosure smell and a likely crash if the repr is huge or unprintable.	names ('overs', 'score', 'wickets', etc.) to formatter functions.
B1 7	LOW	<code>views/match_view.py</code> — <code>JoinMatchView</code> button	The join command (<code>`b!join`</code>) correctly checks <code>`match_instance.lobby_lock`</code> , but the original "Join Match" button created by <code>`b!create`</code> never checks <code>`lobby_lock`</code> at all — a locked lobby can still be joined through the leftover button for up to its 180s timeout.	Add the same <code>`if self.match_instance.lobby_lock: `</code> guard (with an ephemeral rejection message) inside <code>create_button</code> before appending the joining player.
B1 8	LOW	<code>views/toss_view.py</code> — <code>TossView.on_timeout()</code>	Calls <code>`await self.message.edit(...)`</code> once per child inside the <code>`for child in self.children:`</code> loop (so twice, for the two buttons) instead of once after the loop — redundant Discord API calls for an identical edit.	Move the <code>`await self.message.edit(content="Toss timed out.", view=None)`</code> call outside/after the for-loop, matching the pattern already used correctly in <code>YeetMatchView.on_timeout()</code> .
B1 9	LOW	<code>commands/user_i.py</code> — <code>user_installs()</code>	Defines a <code>`/say`</code> app command but the containing function is never called from <code>load.py</code> (only <code>players/host/match/error</code> are wired up), and <code>core/init.py</code> already registers a near-identical <code>`/say`</code> -style DM command directly. This file is dead code that will confuse the next person who edits it.	Delete <code>user_i.py</code> , or if a guild-installable <code>`/say`</code> (as opposed to <code>init.py</code> 's user-installable DM version) is actually wanted, wire <code>`user_installs(bot)`</code> into <code>load.py</code> and give the two commands distinct names.
B2 0	INFO	<code>instance.py</code> — <code>players_queue</code>	<code>self.players_queue = []</code> is declared on every <code>MatchInstance</code> but never read or written anywhere in the codebase.	Either remove it until there's a concrete use (e.g. a substitute-player queue), or repurpose it for the not-out-batsmen queue suggested in §15.2 instead of leaving it dangling.

3. New state to add (MatchInstance is extended, the dict itself is untouched)

`bot.active_matches` keeps mapping `channel_id` → `MatchInstance` exactly as today. Everything below is new attributes set on the `MatchInstance` object (or nested dicts already living on it, like the per-player team dicts from `Game.initialise()`), so the outer dict's shape never changes.

3.1 Per-match fields to add to `MatchInstance.__init__`

```
# --- over / ball state (resets every over) ---
self.balls_this_over      = 0          # legal deliveries bowled so far in current over
(0-5)
self.zeros_this_over      = 0          # dot balls the CURRENT batsman has played this
over
```

```

self.last_over_bowler_id = None      # id of bowler who bowled the previous over (no
consecutive overs)
self.current_over_log      = []      # e.g. ['1','W','0','nb','4','0'] for the live over
display

# --- pending ball state (set while waiting on both picks) ---
self.pending_bowler_pick = None      # int 0-6 once bowler has chosen, else None
self.pending_batsman_pick = None     # int 0-6 once batsman has chosen, else None

# --- AFK tracking (per player id) ---
self.afk_strikes = {}               # {player_id: int} consecutive missed-turn count
self.afk_warned = {}               # {player_id: bool} whether final warning was already sent

# --- innings bookkeeping ---
self.innings_history = []           # list of dicts, one snapshot per completed innings (see 3.2)
self.batting_order_index = {}# {'A': int, 'B': int} index into team_settings batting
order, if you add one
self.declared = False              # batting captain declared the current innings closed
self.follow_on_enforced = False
self.follow_on_available = False  # set True once eligible, captain must opt in

# --- day / session bookkeeping ---
self.overs_bowled_today = 0
self.stumps_called = False        # True between 'end of day' and the host/captain resuming
play

# --- result ---
self.result = None                # already exists – populate with a structured dict, see 9.5

```

3.2 Per-innings snapshot (innings_history entries)

Test cricket needs to compare innings against each other (for the follow-on margin and the final target), so persist a snapshot whenever an innings ends:

```

{
  'innings_no': 1,
  'batting_team': 'A',
  'runs': 312,
  'wickets': 9,
  'overs': 78.4,
  'declared': False,
  'all_out': True,
}

```

3.3 Per-player fields already present that the engine will populate

Game.initialise() already creates these per player — the ball engine is simply the first code path that writes into them: runs, balls, wickets, runs_given, balls_given, is_out, timeline, hattrick, duck, result. Two small additions are useful:

- `balls_faced_legal` — separate from 'balls' if you want strike rate to (correctly) exclude no-balls faced as deliveries but you still want to display them; simplest is to keep 'balls' as legal-balls-faced and add a separate `no_balls_faced` counter only if needed for display.
- `outs_method` — string like 'caught' (numbers matched) or 'timed_out' (4-zero rule) or 'declared_not_out', useful for the scorecard/embed and for stats later.

3.4 Consistency rule to adopt repo-wide (fixes B4/B5 for good)

Pick one convention and apply it everywhere

Recommended: keep `match_instance.players / teamA / teamB / captains` as `discord.Member` OBJECTS (current behaviour), and key every `*stats*` dict (`TeamA/TeamB` from `Game.initialise`, `afk_strikes`, etc.) by integer ID, exactly as `Game.initialise()` already does.

Then: membership checks always compare objects to objects (``x in match_instance.players``), and stat look-ups always go through ``.id`` first (``team_dict[player.id]``). This single rule fixes B4, B5 and prevents the same bug from reappearing in the new game-logic code.

4. Core hand-cricket resolution rule (baseline, before test modifiers)

Both batsman and bowler privately pick a number from 0–6 (via buttons/dropdown, not visible to the other). Once both picks are in:

1. If bowler's number == batsman's number → batsman is OUT (numbers "clash").
2. Otherwise → batsman scores runs equal to their OWN chosen number (0, 1, 2, 3, 4, 5 or 6) added to both the player's and team's total.

Everything below layers the test-format rules on top of this base rule.

5. Ball-by-ball engine — end-to-end flow

This is the centrepiece: a new method, e.g. `Game.play_ball()`, invoked once both picks for the current ball are available. Recommended structure:

5.1 Sequencing the two simultaneous picks

Hand cricket relies on the batsman and bowler choosing blind. Implement this with two short-lived views shown at the same time (or bowler-first-hidden, batsman-second) and only resolve once both `match_instance.pending_bowler_pick` and `match_instance.pending_batsman_pick` are set:

```
async def request_ball(self):
    self.match_instance.pending_bowler_pick = None
    self.match_instance.pending_batsman_pick = None
    # send NumberView to curr_bowler and curr_batsman concurrently
    await asyncio.gather(
        self.prompt_number(self.match_instance.curr_bowler, 'bowler'),
        self.prompt_number(self.match_instance.curr_batsman, 'batsman'),
    )
    # each callback sets pending_*_pick then calls self.try_resolve_ball()

async def try_resolve_ball(self):
    mi = self.match_instance
    if mi.pending_bowler_pick is None or mi.pending_batsman_pick is None:
        return # still waiting on the other player
    async with mi.lock:
        # fixes B8 — atomic resolution
        await self.resolve_ball(mi.pending_bowler_pick, mi.pending_batsman_pick)
```

Using `asyncio.Lock()` here (already declared on `MatchInstance`, currently unused — B8) guarantees a ball can't be double-resolved if both interaction callbacks fire in the same event-loop tick.

5.2 resolve_ball — the core decision tree

```

async def resolve_ball(self, bowler_num: int, batsman_num: int):
    mi = self.match_instance
    bowler = self.teams[mi.bowling_turn][mi.curr_bowler.id]
    batsman = self.teams[mi.batting_turn][mi.curr_batsman.id]

    # ---- 1. NO-BALL CHECK (bowler bowled 0) ----
    if bowler_num == 0:
        await self.handle_no_ball(bowler, batsman)
        return # ball is replayed, over/ball counters untouched, no wicket possible

    # ---- 2. LEGAL DELIVERY: advance ball/over counters first ----
    mi.balls_this_over += 1
    bowler['balls_given'] += 1
    batsman['balls'] += 1

    # ---- 3. WICKET CHECK ----
    if bowler_num == batsman_num:
        await self.handle_wicket(bowler, batsman, method='clash')
    else:
        # ---- 4. RUNS / DOT-BALL (4-ZERO) CHECK ----
        runs = batsman_num
        if runs == 0:
            mi.zeros_this_over += 1
            if mi.zeros_this_over > 4: # 5th dot ball this over
                await self.handle_wicket(bowler, batsman, method='timed_out',
credit_bowler=False)
                mi.current_over_log.append('OUT')
            else:
                self.add_runs(bowler, batsman, runs)
                mi.current_over_log.append('0')
        else:
            self.add_runs(bowler, batsman, runs)
            mi.current_over_log.append(str(runs))

    # ---- 5. CLEAR PENDING PICKS, PUSH TIMELINE, CHECK OVER/INNINGS END ----
    batsman['timeline'].append(mi.current_over_log[-1])
    mi.pending_bowler_pick = None
    mi.pending_batsman_pick = None
    await self.after_ball_checks()

```

5.3 Helper: add_runs

```

def add_runs(self, bowler: dict, batsman: dict, runs: int):
    mi = self.match_instance
    batsman['runs'] += runs
    bowler['runs_given'] += runs
    self.teams[mi.batting_turn]['runs'] += runs # team aggregate (see B9 re-org)
    mi.score += runs

```

6. No-ball rule (bowler bowls 0)

Test-format-specific rule: a bowler is not allowed to choose 0. If they do, it's a no-ball.

6.1 Effects of a no-ball

- +1 run is added immediately to the batting team's total (and conventionally to the bowler's runs_given, not the batsman's personal tally) — this is an extra, not a scoring shot.
- The ball does NOT count toward the over (mi.balls_this_over is not incremented) — the same bowler must bowl again to complete the legal six.
- No wicket can fall off a no-ball under the clash rule (batsman cannot be given out on this delivery) — mirrors real cricket's free-hit-adjacent protection. The 4-zero rule also does not apply, since the batsman never got to choose a number against a legal ball.
- Track repeated no-balls per bowler (mi-level counter or bowler['no_balls']) — useful for stats and for an optional house rule such as "3 no-balls in one over = bowler is changed automatically", if you want extra punishment for stalling bowlers.

6.2 handle_no_ball implementation

```
async def handle_no_ball(self, bowler: dict, batsman: dict):
    mi = self.match_instance
    bowler['runs_given'] = bowler.get('runs_given', 0) + 1
    bowler['no_balls'] = bowler.get('no_balls', 0) + 1
    self.teams[mi.batting_turn]['runs'] += 1
    mi.score += 1
    mi.current_over_log.append('nb')
    mi.pending_bowler_pick = None
    mi.pending_batsman_pick = None
    await self.ctx.send(f"No ball! {bowler1_mention} cannot bowl a 0. +1 run. Ball
replayed.")
    await self.request_ball() # immediately re-prompt the same bowler/batsman, over
unchanged
```

7. Four-dot-ball rule (batter "timed out" for over-blocking)

A batsman may play at most four 0s (dot balls) within a single over. The 5th time they choose 0 in that same over, they are out — regardless of what the bowler chose, since the comparison never matters once the count is exceeded.

7.1 Key implementation details

- The counter (mi.zeros_this_over) is scoped to the CURRENT OVER and the CURRENT BATSMAN on strike — reset it to 0 every time an over completes AND every time a new batsman comes to the crease mid-over (a fresh batsman should get a fresh count, not inherit the previous batsman's dots).
- Only legal deliveries count toward this limit — a no-ball never reaches this branch (see §6), so it cannot contribute to the 4-zero count.
- The 5th dot ball still needs to advance balls_this_over and the bowler's balls_given (it was a legal, fairly bowled delivery) — only the batsman is dismissed, the ball itself is completely normal from the bowler's perspective. This is reflected in §5.2 where the increment happens before the dot-ball branch.
- Credit the wicket to the rule, not the bowler: do not increment bowler['wickets'] for a timed-out dismissal (it wasn't a clash) — pass credit_bowler=False as shown in §5.2 so bowling figures stay accurate.

7.2 Resetting the counter correctly

```
# Reset on every NEW batsman arriving at the crease (wicket or end of previous batsman's
spell)
def on_new_batsman_in(self):
    self.match_instance.zeros_this_over = 0

# Reset at the start of every new over (see section 8)
def on_new_over(self):
    self.match_instance.balls_this_over = 0
    self.match_instance.zeros_this_over = 0
    self.match_instance.current_over_log = []
```

8. Over completion & bowler rotation

8.1 Detecting the end of an over

```
async def after_ball_checks(self):
    mi = self.match_instance
    if await self.check_wicket_just_fell_and_handle_next_batsman():
        return # next_batsman() flow will call back into request_ball() once selection is
done

    if mi.balls_this_over >= 6:
        await self.complete_over()
        return

    if not await GameChecks.game_continue_check(mi):
        await self.end_innings()
        return

    await self.request_ball() # same over continues, prompt for the next ball
```

8.2 complete_over

```
async def complete_over(self):
    mi = self.match_instance
    mi.overs += 1 # whole overs bowled this innings
    mi.overs_bowled_today += 1
    mi.last_over_bowler_id = mi.curr_bowler.id
    await self.ctx.send(self.render_over_summary()) # uses mi.current_over_log
    self.on_new_over() # reset balls_this_over / zeros_this_over / log

    if not await GameChecks.game_continue_check(mi):
        await self.end_innings()
        return

    await self.next_bowler() # dropdown excludes mi.last_over_bowler_id (fixes
B6)
    # next_bowler's callback should call self.request_ball() once a bowler is confirmed
```

Note: classic limited-overs cricket caps how many overs a single bowler can bowl in a match (overs/5), but TEST cricket has no such per-bowler over cap — only the no-consecutive-overs rule applies. Don't port a one-day over-limit into this test-format build unless you want it as a house rule.

9. Innings & match lifecycle

9.1 All-out (last-man-standing) condition

Hand-cricket teams don't bat in pairs the way real cricket does (only one `curr_batsman` is ever selected) — but you can still respect the spirit of "last man stands" so a team isn't reduced to 0 batsmen. Recommended: an innings ends when `wickets == squad_size - 1` (one undischarged batsman remains and has no partner left), not `squad_size`.

```
def is_all_out(self, mi) -> bool:
    batting_team_size = len(mi.teamA if mi.batting_turn == 'A' else mi.teamB)
    return mi.wickets >= batting_team_size - 1
```

9.2 Reasons an innings can end (any one of)

- All out (§9.1).
- Overs allotted for the innings/day are exhausted (`mi.overs >= mi.match_settings['overs']`).
- Batting captain declares (§10).
- Chasing team passes the target in innings 2 or 4 (match-winning condition, not just innings-end).
- Days remaining run out (`crr_day > match_settings['days']`) — forces a draw, see §9.5.

9.3 end_innings — transition logic

```
async def end_innings(self):
    mi = self.match_instance
    mi.innings_history.append(self.snapshot_current_innings())

    if mi.innings == 1:
        await self.start_next_innings(swap_batting=True)
    elif mi.innings == 2:
        # decide normal swap vs follow-on (see section 11)
        if self.is_follow_on_eligible():
            mi.follow_on_available = True
            await self.offer_follow_on()          # waits on bowling captain's decision
        else:
            mi.target = self.compute_target()    # see 9.4 below; only meaningful once both
            teams have batted once
            await self.start_next_innings(swap_batting=True)
    elif mi.innings == 3:
        mi.target = self.compute_target()
        await self.start_next_innings(swap_batting=not mi.follow_on_enforced)
    elif mi.innings == 4:
        await self.finalise_result()
```

9.4 game_continue_check rewrite (fixes B1/B2/B3)

Replace the flat list-of-booleans with explicit short-circuited checks that also produce a reason, so `finalise_result()` doesn't have to re-derive why the match stopped:

```
@staticmethod
async def game_continue_check(mi) -> bool:
```

```

    if not mi.game_started:
        return False
    if mi.innings > 4:
        return False
    if mi.crr_day > mi.match_settings['days']:
        mi.result = mi.result or {'type': 'draw', 'reason': 'days_exhausted'}
        return False
    if mi.innings in (2, 4) and mi.target is not None and mi.score >= mi.target:
        mi.result = {'type': 'win', 'winner': mi.batting_turn, 'reason': 'target_chased'}
        return False
    if mi.overs >= mi.match_settings['overs']:
        return False # innings-level stoppage, NOT necessarily match over – caller
checks innings count separately
    if Game.is_all_out_static(mi):
        return False
    return True

```

Key fix versus the original: target is only ever read after it's been assigned (innings 2/4), and the trailing bare match_instance entry (B1) is removed entirely.

9.5 Result determination (finalise_result)

```

async def finalise_result(self):
    mi = self.match_instance
    last_innings = mi.innings_history[-1]
    if mi.score >= mi.target:
        wickets_left = (squad_size - 1) - mi.wickets
        mi.result = {'type': 'win', 'winner': mi.batting_turn, 'margin': f'{wickets_left}
wickets'}
    elif mi.crr_day > mi.match_settings['days']:
        mi.result = {'type': 'draw'}
    else:
        runs_short = mi.target - mi.score - 1
        bowling_team = 'B' if mi.batting_turn == 'A' else 'A'
        mi.result = {'type': 'win', 'winner': bowling_team, 'margin': f'{runs_short}
runs'}
    await self.ctx.send(self.render_scorecard_embed())

```

10. Declaration

The batting captain may close their own innings at any point while batting, before being all out, to set up time to bowl the opposition out.

10.1 Rules

- Only the captain of the CURRENTLY BATTING team can declare (mi.batting_turn captain).
- Cannot declare on innings 4 (the team chasing a target has no reason to and shouldn't be able to shortcut the match).
- Conventionally disallowed with fewer than some minimum number of overs faced or before a minimum score, if you want to prevent trivial/troll declarations at 0/0 — make this configurable in match_settings (e.g. min_declare_overs) rather than hardcoded.
- Declaring immediately triggers end_innings() exactly like an all-out, just with all_out=False, declared=True recorded in the innings snapshot.

10.2 Command sketch

```
@bot.command(name='declare', aliases=['dec'])
async def declare(ctx):
    mi = ctx.bot.active_matches.get(ctx.channel.id)
    if not mi or not mi.game_started:
        return await ctx.send('No match in progress.')
    batting_captain = mi.teamA_captain if mi.batting_turn == 'A' else mi.teamB_captain
    if ctx.author != batting_captain:
        return await ctx.send('Only the batting captain can declare.')
    if mi.innings == 4:
        return await ctx.send('Cannot declare in the final innings.')
    mi.declared = True
    await ctx.send(f'{batting_captain.mention} has DECLARED the innings closed at
{mi.score}/{mi.wickets}!')
    game = Game(ctx, mi)          # or fetch the live Game instance if you keep one cached
    await game.end_innings()
```

11. Follow-on

If the team batting second is dismissed (or declares) well behind the first team's total, the first team's captain may enforce the follow-on, forcing the second team to bat again immediately instead of the sides alternating normally.

11.1 Eligibility

```
def is_follow_on_eligible(self) -> bool:
    mi = self.match_instance
    if mi.innings != 2:
        return False
    innings1 = mi.innings_history[0]
    lead = innings1['runs'] - mi.score      # team 1's score minus team 2's score
    margin = mi.match_settings.get('follow_on_margin', 150) # make configurable per overs
    format
    return lead >= margin
```

150 runs mirrors a common multi-day cricket convention but should be exposed as a settable `match_settings` value (e.g. scaled to overs/team-size) since this is hand cricket, not 11-a-side first-class cricket — set a sensible default and let the host tune it with `set_overs`-style command.

11.2 Enforcing or declining

```
async def offer_follow_on(self):
    mi = self.match_instance
    captain = mi.teamA_captain if mi.bowling_turn == 'A' else mi.teamB_captain # team
    that bowled 2nd innings, i.e. batted 1st
    await self.ctx.send(
        f'{captain.mention}, your team leads by {self.current_lead()} runs. '
        f'Enforce the follow-on? Use `{prefix}followon yes` or `{prefix}followon no`.'
    )
    # on 'yes': mi.follow_on_enforced = True; batting_turn stays the team that just batted
    # on 'no' or timeout: normal alternation – team that led now bats again
```

11.3 Effect on the innings counter

- Innings count still increments normally (1 → 2 → 3 → 4); what changes is WHICH team bats in innings 3.
- If enforced: innings 3 batting_turn == innings 2's batting_turn (team 2 again). If declined or not eligible: innings 3 batting_turn == innings 1's batting_turn (normal alternation, team 1 again).
- `mi.target` is only computed once both teams have batted at least once each (i.e. after innings 3 in a follow-on scenario, or after innings 2 normally is too early — target is really only meaningful entering the FINAL innings). Compute it right before the last innings starts, as shown in §9.3.

12. AFK / inactivity handling

Every point where the engine blocks on a human action (number pick, batsman select, bowler select, follow-on decision, declaration window) needs a timeout so one missing player can't freeze the whole match.

12.1 Strike-based system

- Each prompt (NumberView / PlayerView / decision views) gets a bounded `discord.ui.View(timeout=N)` — 30–45s for a number pick is reasonable for a quick-format game.
- On `on_timeout()`: increment `mi.afk_strikes[player.id]`, then branch on the count.
- Strike 1: send a warning ping, then auto-substitute a value so the over isn't blocked — e.g. auto-pick a number for the missing player (random 0–6 for fairness, or a fixed conservative value like 1, your call) and resolve the ball as normal.
- Strike 2 (or N, make it configurable): same as strike 1, but flag the over summary so spectators see who's stalling.
- Strike 3+: treat the AFK player as effectively unavailable for the rest of their spell — if they're the batsman, give them out (`out_method='afk_timeout'`, does not count as a fielding wicket for the bowler exactly like the 4-zero rule); if they're the bowler, force a captain to pick a new bowler immediately (re-trigger `next_bowler` with the AFK bowler excluded).
- Reset `afk_strikes[player.id]` back to 0 once that player successfully responds in time — strikes shouldn't accumulate forever across an entire multi-day match, only count consecutive misses.

12.2 AFK during selection (captain picking next batsman/bowler)

If a captain themselves goes AFK during PlayerView selection, apply the same strike system to the captain, and after the final strike auto-pick the next available player in batting/bowling order (e.g. first not-out player by join order) so the game can keep moving without the captain.

12.3 Implementation sketch

```
async def on_timeout(self): # inside a NumberView
    mi = self.match_instance
    mi.afk_strikes[self.player.id] = mi.afk_strikes.get(self.player.id, 0) + 1
    strikes = mi.afk_strikes[self.player.id]
    max_strikes = mi.match_settings.get('afk_max_strikes', 3)

    if strikes >= max_strikes:
        await self.channel.send(f'{self.player.mention} has been AFK too many times and is removed from this turn.')
```

```

        await self.game.handle_afk_removal(self.player, self.role)    # role = 'batsman' |
'bowler'
    else:
        auto_value = random.randint(0, 6)
        await self.channel.send(f'{self.player.mention} did not respond in time (strike
{strikes}/{max_strikes}). Auto-picked {auto_value}.')
        await self.game.submit_pick(self.role, auto_value)

```

13. Multi-day / stumps handling

- Each day has an overs quota (e.g. `match_settings['overs_per_day']`); track `mi.overs_bowled_today` and call `call_stumps()` once it's reached, independent of the innings-level overs counter.
- `call_stumps()`: set `mi.stumps_called = True`, persist state to Turso (see §15.4), and stop prompting for new balls until the host/captain issues a `resume_play` (or next-day) command.
- On resume: `mi.curr_day += 1`, `mi.overs_bowled_today = 0`, `mi.stumps_called = False`, continue the SAME over/innings state exactly where it left off (`balls_this_over`, `zeros_this_over`, `curr_batsman`, `curr_bowler` all carry over — nothing about a day boundary resets player state, only the daily overs counter resets).
- Day exhaustion (`mi.curr_day > mi.match_settings['days']`) is a draw trigger handled in §9.4 — no winner is declared, regardless of where the chasing team's score sits relative to the target.

14. Concurrency & data-integrity recommendations

- Use `mi.lock` (already declared, currently dead code — B8) around the entire `resolve_ball` critical section, and again around `end_innings/transition` code, so two interactions firing in the same tick (e.g. a double-click, or a slow network re-delivering a Discord interaction) cannot double-apply a ball or double-trigger an innings change.
- Guard against duplicate interaction callbacks: once a `NumberView` button is pressed, immediately call `button.disabled = True / view.stop()` and edit the message before doing any game-state mutation, so a second rapid click can't re-enter the handler.
- Use `interaction.response.defer()` consistently before any awaited game-logic call (already done in the existing views) so Discord doesn't time out the interaction (3 second ack window) while `resolve_ball` is awaiting database writes.
- Never mutate `match_instance` from two different coroutines without the lock — particularly the bowler-pick and batsman-pick callbacks, which legitimately race by design (that's the point of blind picks) and must funnel into the same locked `try_resolve_ball()`.

15. Performance recommendations

15.1 Discord API usage

- Prefer `message.edit()` over `ctx.send()` for anything that updates in place (live scoreboard, current over ticker) — sending a brand-new message every ball will both clutter the channel and burn through Discord's per-channel rate limit far faster than necessary edits.

- Batch the 'ball result' + 'updated score' + 'over progress' into a single embed edited once per ball instead of three separate `ctx.send()` calls — cuts API calls roughly 3x per ball, which matters a lot over a 90-over innings.
- Cache the live scorecard embed/message reference on the Game instance (not re-fetched from Discord every time) so updates are $O(1)$ edits, not a fetch+edit.
- Use ephemeral interaction responses for the number-pick prompts (only the picking player needs to see their own buttons) instead of public messages — reduces channel noise and the number of messages Discord has to fan out to all members.

15.2 In-memory data structures

- `next_batsman()` currently rebuilds a fresh `not_out_players` dict via comprehension on every call — fine at hand-cricket squad sizes (well under a few dozen), not a real bottleneck, but if you want to avoid the rebuild, maintain a maintained queue (`collections.deque`) of not-yet-out batsmen and pop from it instead of filtering the whole dict each time.
- `TeamA['name']/'runs'/'balls'/'wickets'` co-mingled with player rows (B9) forces every consumer to `isinstance-check` keys; separating aggregate stats into their own dict removes that branch from every iteration site (`PlayerSelect`, scorecard rendering, `after_ball_checks`), which is a small but easy win since these structures are read on literally every ball.
- `current_over_log` should be a plain list reset each over (cheap), but the per-player timeline `deque(maxlen=6)` is already correctly sized — just make sure you push to it inside `resolve_ball` (it currently sits unused/empty after `initialise()`).

15.3 Avoiding redundant `game_continue_check` calls

`game_continue_check()` is currently called multiple times per loop iteration in `continue_game()` (lines 103 and 107 of `game.py`) doing essentially the same evaluation twice in a row for no reason. Once it's rewritten per §9.4, call it exactly once per logical checkpoint (after each ball, after each over, after each innings) and branch on the single result rather than re-invoking it defensively.

15.4 Database (Turso) writes

- `TursoDB.init_match()` exists but `db.py` is referenced from `db.execute` per-call with `httpx.AsyncClient` — don't write to Turso on every single ball (90 overs × 2 innings × multiple days = thousands of round trips for a single match). Persist match state at coarser checkpoints: end of over, end of innings, stumps, and match end — that's a few dozen writes per match instead of thousands, while still giving you crash recovery.
- If you want ball-level durability without per-ball latency, buffer the last N balls in memory (`mi.current_over_log` already does this for the active over) and only flush to Turso at over boundaries — you lose at most one partial over's history on a hard crash, which is an acceptable tradeoff for a Discord game bot.
- `schemas.sql`'s `ongoing_matches` table stores `match_state/match_settings/match_data` as JSONB — serialize exactly the new fields from §3 into `match_state` on each checkpoint write so a restart can fully rehydrate `balls_this_over`, `zeros_this_over`, `innings_history`, etc., not just score/wickets.

15.5 Avoid blocking/long-running work on the event loop

- `TursoDB` uses `httpx.AsyncClient` correctly (non-blocking) — keep all future I/O (DB, any future HTTP calls) `async`; never introduce requests (`sync`) into a `discord.py` bot, it will freeze the whole bot for every guild during the call.

- Keep `resolve_ball()`'s critical section (inside the lock) limited to pure in-memory state mutation; do any Discord message edits / DB writes either just outside the lock or fire as a follow-up task, so the lock is held for microseconds, not for the duration of a network call.

16. Suggested implementation roadmap

Phased so each phase is independently testable in a Discord channel before moving on.

Phase 0 — Stabilise

- Rotate the leaked Discord bot token and Turso credentials immediately (see security note at the top of this document) — do this before anything else, independent of the code fixes below.
- Fix B1–B9 and B10–B20 (Section 2). Nothing in later phases works reliably until `game_continue_check`, `start_game`, the membership checks, the config import, and Turso persistence are correct.
- Adopt the object-vs-id convention from §3.4 across the whole codebase in one pass — it's the root cause of B4, B5, and B15.

Phase 1 — Core ball engine

- Add the new `MatchInstance` fields (§3.1).
- Build the `NumberView` (0–6 picker) for bowler and batsman.
- Implement `resolve_ball` / `add_runs` / `handle_wicket` for the BASE rule only (no test-format modifiers yet) and get a full over working end-to-end with live score updates.

Phase 2 — Test-format modifiers

- Layer in no-ball handling (§6).
- Layer in the 4-zero rule (§7), including correct reset-on-new-batsman behaviour.
- Layer in over completion + bowler rotation, excluding the last over's bowler (§8).

Phase 3 — Innings & match lifecycle

- Rewrite `game_continue_check` (§9.4) and wire `end_innings/start_next_innings/finalise_result` (§9.3, §9.5).
- Add the `declare` command (§10).
- Add follow-on eligibility + offer/enforce flow (§11).

Phase 4 — Resilience

- AFK strike system across all blocking prompts (§12).
- Day/stumps handling (§13) and the `resume-play` command.
- Concurrency hardening — actually use `mi.lock` everywhere it's needed (§14).

Phase 5 — Polish & performance

- Switch live updates to `message.edit()` with a single cached embed (§15.1).
- Move Turso writes to checkpoint-only persistence (§15.4).

- Scorecard/result embed, timeline rendering, over-by-over ticker.

Bottom line

Rotate your bot token and Turso credentials first — they were exposed in the uploaded archive.

The lobby/toss/team-management layer is solid and doesn't need a rewrite, but it has 20 real bugs (Section 2) sitting in it, several of which (B1, B2, B10, B11) make the bot crash or silently never start a match today.

The entire scoring engine (Sections 4-9) is the real build — nothing currently reads a number from either player.

Fix Section 2's bugs first; several of them will make the new engine appear broken even when the new code is correct, because today's start/continue-check path is unconditionally wrong.

Every new field proposed lives on MatchInstance or its nested per-player dicts — `bot.active_matches` stays a plain `{channel_id: MatchInstance}` dict throughout, as required.